Credit Card Fraud Detection System

The Complete Technical & Conceptual Project Guide — Built from First Principles in Pure NumPy

---

Course: Python for Data Science (PDS)	**Dataset:** Kaggle Credit Card Fraud (284,807 txns)
Implementation: 100% Pure NumPy (Zero sklearn/ML packages)	**Date:** October 2026
Authors: Student Project Engineering Team	**Status:** 10 Milestones Verified via 12 Monte Carlo Runs

Hey there! Welcome to our complete project guide.
If you are reviewing this code or working with us, you don't need a PhD in statistics to follow along. This document explains **what the dataset actually is**, **why normal machine learning fails horribly** on fraud detection, **how we solved it step-by-step from scratch**, and **what the final 12-run benchmark proved**.

1. The Dataset Explained (For Anyone New to It)

The dataset contains real-world credit card transactions made by European cardholders in September 2013 across 2 days. There are **284,807 transactions** in total, but only **492 are fraudulent** (a tiny 0.1727% fraud rate).

The Naive Accuracy Trap:

If you write a simple 1-line script that answers *'NO FRAUD'* for every single transaction, it achieves a **99.827% accuracy rate**! In a school exam, 99.8% looks like an A+. But in a bank, that model just let all 492 criminals steal hundreds of thousands of dollars. **Therefore, in fraud detection, Accuracy is meaningless.** We only evaluate: **Recall** (what % of frauds did we stop?), **Precision** (how many false alarms did we raise?), and **Dollar Value Recall** (what % of stolen cash was saved?).

Why are features V1 through V28 anonymized?

Banks cannot publicly share customer names, credit card numbers, or store locations due to privacy laws. The researchers transformed the raw confidential features using **Principal Component Analysis (PCA)** — a mathematical rotation that projects high-dimensional data onto 28 numerical coordinates (V_1 to V_{28}). These coordinates are centered around 0 and completely de-correlated. Only **Time** and **Amount** remain in their original raw form.

2. Real-World Banking Economics (The Dollar Problem)

Standard machine learning algorithms treat all errors identically (1 missed fraud = 1 false alarm). In banking, this is financially disastrous:

- **Cost of a False Alarm (False Positive = $2.50):** When a genuine card is declined, the bank sends an automated SMS or phone verification. As established in benchmark banking research by *Bahnsen et al. (2015)* and *Dal Pozzolo et al. (2014)*, the administrative labor and notification overhead is approximately **$2.50 per false alert**.

- **Cost of a Missed Fraud (False Negative = Transaction Amount):** Under credit card protection laws (Regulation E), innocent customers are not liable for stolen charges. The bank must reimburse the cardholder 100% out of its own capital. Missing a $4,000 fraud costs the bank exactly $4,000.
- **The 'Card Testing' Scam:** Over 53% of all fraud incidents in this dataset are micro-transactions under $10 (fraudsters buying $0.99 app credits to test if stolen card numbers work). Investigating all micro-alerts costs $2.50 in human labor to protect less than $1.00 of risk!

3. Our 10-Step Technical Journey (Built 100% from Scratch)

Every single algorithm, feature transformer, gradient descent loop, and evaluation metric was coded strictly in pure Python and NumPy — with zero calls to scikit-learn, xgboost, or torch.

Step 0: Pristine Baseline Model

Notebook: notebooks/fraud_detection.ipynb

Implemented pure Logistic Regression using vectorized gradient descent and Bernoulli log-loss. Balanced classes via random undersampling. Kept permanently frozen as an untouched safety baseline (tagged baseline-v1.0 in Git).

Step 1: Feature Engineering & SMOTE

Notebook: notebooks/step1_smote_and_features.ipynb

Mapped Time onto a 24-hour diurnal clock using Sin and Cos transformations to mirror human sleep/wake cycles. Log-transformed Amount to tame extreme outliers. Built vectorized SMOTE from scratch using k-NN Euclidean distance to create synthetic fraud instances between minority neighbors. Slashed false alarms by 28%.

Step 2: Deep Neural Network (MLP)

Notebook: notebooks/step2_neural_net_scratch.ipynb

Constructed a 2-hidden layer Multi-Layer Perceptron (31 -> 32 -> 16 -> 1) in pure NumPy. Used He (Kaiming) normal weight initialization to prevent dying neurons, ReLU activations, vectorized backpropagation matrix calculus, and classical momentum (beta=0.9). Reached 89.2% recall on non-linear patterns.

Step 3: Stratified 5-Fold CV & Cost Matrix

Notebook: notebooks/step3_cost_matrix_kfold.ipynb

Implemented Stratified 5-Fold cross-validation from scratch to prevent data leakage. Formulated the Banking Dollar Cost Matrix (FN = Amount, FP = $2.50). Proved that shifting the decision threshold from tau=0.50 to tau*=0.83 cut false alarms by 83.5% and saved $44,323 in net fraud dollars.

Step 4: Gradient Boosted Trees (GBDT)

Notebook: notebooks/step4_gbdt_scratch.ipynb

Engineered an iterative ensemble of 30 regression trees trained on pseudo-residuals (negative gradients r_i = y_i - p_i). Leaf values computed via second-order Newton-Raphson approximation. Achieved 91.84% recall. Extracted variance-reduction feature importances from scratch: Feature V14 alone drove 77.2% of all splits!

Step 5: Unsupervised Isolation Forest

Notebook: notebooks/step5_isolation_forest_scratch.ipynb

Built an anomaly detection tree ensemble requiring ZERO training labels. Randomly partitions space; anomalies require very few recursive cuts to isolate. Calibrated path lengths with Euler-Mascheroni constant c(n). Scored 0.9540 ROC-AUC with zero supervision, providing critical defense against zero-day fraud attacks.

Step 6: Amount-Weighted Cost-Sensitive Loss

Notebook: notebooks/step6_cost_sensitive_loss_scratch.ipynb

Formulated an Amount-Weighted binary cross-entropy loss where fraud sample weights scale logarithmically with transaction value: c_i = 1 + kappa*ln(Amount + 1). Derived exact analytical gradient updates via chain rule. Achieved 96.00% Dollar Value Recall ($10,085 stolen capital protected).

Step 7: Hybrid Soft-Voting Ensemble

Notebook: notebooks/step7_hybrid_ensemble_scratch.ipynb

Fused 4 distinct mathematical paradigms (Linear, Neural, Tree, Anomaly) into a soft-voting convex combination on the unit simplex. Error covariance dropped drastically, producing the highest overall ROC-AUC of 0.9821 across all models.

Step 8: Dynamic Cost Thresholding tau*(x)

Notebook: notebooks/step8_instance_dependent_thresholding.ipynb

Derived Bayesian minimum risk dynamic thresholding: tau*(x) = C_FP / (C_FP + Amount). A $5,000 transfer alerts at tau*=0.0005, while a $1.50 micro-charge requires tau*=0.625. Stratified into 5 financial tiers, proving that dynamic thresholding slashes micro-tier false alarms by 87%.

Step 9: RetinaNet Focal Loss from Scratch

Notebook: notebooks/step9_focal_loss_scratch.ipynb

Implemented Focal Loss FL(p_t) = -alpha*(1 - p_t)^gamma * ln(p_t) with analytical chain-rule gradients. For gamma=2.0, well-classified genuine transactions have their gradients silenced by 10,000x, allowing gradient descent to focus 100% on hard, borderline frauds (68.27% F1-score with only 66 false alarms).

Step 10: Stacked Generalization (Meta-Learner)

Notebook: notebooks/step10_stacked_generalization_scratch.ipynb

Implemented Wolpert's (1992) Stacking. Used 5-fold out-of-fold (OOF) cross-validation to generate leak-free meta-features. Trained a Level-1 meta-classifier that learned non-linear gating over all base models. Boosted Precision to 42.16% and F1-score to 56.95% while cutting false alarms down to just 118.

4. The 12-Run Monte Carlo Benchmark (The Statistical Proof)

To guarantee that our results were not accidental flukes or lucky splits, we executed a dedicated benchmark (benchmark_12_runs.py) across **12 independent pseudo-random seeds** ([42, 101, 7, 999, 2026, 13, 88, 555, 1234, 777, 314, 99]). Each run re-partitioned the 284,807 transactions, retrained the models from scratch, and evaluated on ~56,961 held-out test transactions.

Model Architecture	Event Recall	Dollar Recall	Precision	F1-Score	ROC-AUC	Avg $ Saved	FP	FN
1. Baseline LogReg	86.7% ± 3.5%	77.1% ± 11.3%	13.6% ± 1.8%	23.4% ± 2.7%	0.9770	$9,378	551	13.0
2. Cost-Sensitive LogReg	**94.7% ± 1.9%**	**93.6% ± 6.5%**	1.2% ± 0.2%	2.4% ± 0.3%	0.9751	**$11,343**	7596	**5.2**
3. Neural Net (MLP)	89.2% ± 3.0%	81.9% ± 9.3%	4.9% ± 0.4%	9.4% ± 0.8%	0.9762	$9,971	1692	10.6
4. GBDT Classifier	87.9% ± 3.2%	79.8% ± 9.1%	8.8% ± 1.7%	15.9% ± 2.8%	0.9792	$9,697	930	11.8
5. Isolation Forest (Unsupervised)	65.9% ± 9.4%	58.9% ± 14.4%	6.8% ± 2.1%	12.3% ± 3.4%	0.9459	$7,224	945	33.4
6. Soft-Voting Ensemble	89.7% ± 3.0%	83.3% ± 9.7%	6.2% ± 0.6%	11.5% ± 1.0%	**0.9821**	$10,164	1351	10.1
7. Stacked Meta-Learner (Step 10)	87.8%	92.7%	**42.2%**	**56.9%**	0.9798	$9,742	**118**	12.0

5. Executive Decision: Which Model Should You Deploy?

Depending on the bank's operational strategy, the optimal model changes:

- **Deploy Cost-Sensitive Model (Step 6):** When the mandate is 'Protect Capital at All Costs'. It recovered 93.57% of all stolen cash and missed only 5 frauds per run, saving $11,343 on average.
- **Deploy Stacked Meta-Learner (Step 10):** When the mandate is 'Operational Efficiency & Low False Alarms'. Slashed false alarms to just 118 while maintaining a massive 56.95% F1-score and protecting $9,742 in stolen cash.
- **Run Isolation Forest (Step 5) in Parallel:** As an unsupervised safety net. Because it uses zero labels, it detects zero-day attacks and novel fraud strategies that bypass supervised rules.

6. Project Repository Map

All code is mirrored across your IDE workspace and C:\Users\HP\Desktop\PDS:

File Path	Description & Milestone
notebooks/fraud_detection.ipynb	Untouched baseline submission (Logistic Regression from scratch)
notebooks/step1_smote_and_features.ipynb	Step 1: SMOTE + 24h Sin/Cos Time + Log-Amount
notebooks/step2_neural_net_scratch.ipynb	Step 2: 2-Hidden-Layer Neural Network (MLP) with Momentum
notebooks/step3_cost_matrix_kfold.ipynb	Step 3: Stratified 5-Fold Cross-Validation + Banking Cost Matrix
notebooks/step4_gbdt_scratch.ipynb	Step 4: Gradient Boosted Decision Trees (GBDT, 30 trees)
notebooks/step5_isolation_forest_scratch.ipynb	Step 5: Unsupervised Anomaly Isolation Forest

notebooks/step6_cost_sensitive_loss_scratch.ipynb	Step 6: Amount-Weighted Cost-Sensitive Loss Function
notebooks/step7_hybrid_ensemble_scratch.ipynb	Step 7: Multi-Paradigm Soft-Voting Ensemble (0.9821 ROC-AUC)
notebooks/step8_instance_dependent_thresholding.ipynb	Step 8: Dynamic Bayesian Cost Thresholding tau*(x)
notebooks/step9_focal_loss_scratch.ipynb	Step 9: RetinaNet Focal Loss & Hard Example Mining
notebooks/step10_stacked_generalization_scratch.ipynb	Step 10: Stacked Generalization & Out-Of-Fold Meta-Learner
notebooks/fraud_detection_master_capstone.ipynb	Unified Master Capstone Notebook combining all techniques
benchmark_12_runs.py	12-Seed Monte Carlo experimental stability evaluation script
benchmark_12_runs_summary.csv	Raw CSV output containing statistical means and standard deviations

All code and experiments were constructed 100% from first principles in pure NumPy. Every single step is modular and can be verified independently in its respective notebook.